COMPTIA PENTEST+ · PT0-003 · DEEP-DIVE STUDY GUIDE

Objectives 4.3 & 4.5 — Authentication & Web Application Attacks

The two densest objectives, broken into a handful of memorable families. Each attack: what it is, how it works (with a recognizable example), the official tool, and the fix.

The two objectives worth the most study time

These two objectives carry the most sub-items in Domain 4 (35% of the exam). The trick to mastering them is **not** memorizing a flat list — it's grouping each into a small number of *families* that share one mechanism, then learning the family. Every attack below is tagged with its official tool, and each family ends with a **REMEMBER** hook.

> **How the two overlap:** several attacks live on the border of both objectives — *brute-force*, *session* handling, *JWT*, and *API* abuse are authentication problems *and* web problems. Learn them once; expect them in either context.

4.3 — Authentication Attacks

perform authentication attacks using the appropriate tools

MEMORY SPINE: GUESS IT → CRACK IT → STEAL IT → ABUSE THE PROTOCOL

Authentication = proving you're who you claim. Every attack either *obtains the secret* (guess / crack / steal) or *abuses the protocol that checks it*.

1 · GUESS IT — online password attacks

Run against a *live* login, so they're subject to lockout, rate-limiting, and detection.

Attack	How it works	Tool(s)	Fix
Dictionary	Try a wordlist of likely passwords (e.g., `rockyou.txt`) against the login.	Hydra, Medusa	Lockout, MFA, ban common passwords.
Brute-force	Try every possible combination — defeated by password *length*.	Hydra, Medusa	Lockout, length, MFA.
Password spraying	ONE common password across MANY users to stay *under* the lockout threshold.	CrackMapExec, Hydra	MFA, smart lockout, ban common passwords.
Credential stuffing	Replay `user:pass` pairs leaked in *other* breaches (password reuse).	Scripts, CME	MFA, breached-password screening.

REMEMBER: online = subject to lockout. **SPRAYING'S WHOLE PURPOSE IS LOCKOUT-EVASION** — one password, many users (the opposite of brute-forcing one user).

2 · CRACK IT — offline hash attacks

Once you *have the hashes* (from a dump, capture, or file), cracking is offline: no server, no lockout, GPU speed.

Mode / concept	How it works	Tool(s)	Notes
Straight / dictionary	Hash each wordlist entry and compare to the target (`-a 0`).	hashcat, John	Fastest first pass.
Rule-based	Mutate the wordlist (`Password` → `P@ssw0rd!`) with rules.	hashcat, John	Catches predictable "complexity."
Mask attack	Brute-force over a *known pattern* to shrink the keyspace, e.g. `?u?l?l?l?l?d?d?d?d?s` (Ssss + 4 digits + symbol) (`-a 3`).	hashcat	"Smart" brute-force.
Hybrid	Combine wordlist + mask (`-a 6` / `-a 7`).	hashcat	Word + appended pattern.

Where the hashes come from: NTLM (SAM/LSASS dump) · `/etc/shadow` · NetNTLMv2 (captured by Responder) · Kerberos TGS (from Kerberoasting).

REMEMBER: HASHCAT = GPU SPEED, JOHN = VERSATILE. No server means no lockout — which is exactly why the defense for a stolen hash DB is *strong, slow hashing* (bcrypt/argon2), not lockout.

3 · STEAL IT — reuse credential material (Windows/AD)

AD lets the *secret material itself* authenticate you — so stealing it means becoming the user, no cracking required.

Attack	How it works	Tool(s)	Fix
Pass-the-Hash	Use the NTLM *hash* directly in NTLM challenge/response — no plaintext.	CME, Impacket*	Credential Guard, LAPS, limit local-admin reuse.
Pass-the-Ticket	Inject a stolen/forged Kerberos *ticket* into your session.	Mimikatz*, Rubeus*	Protect DCs, rotate krbtgt, monitor.
Pass-the-Token	Replay a stolen session / OAuth / access *token*.	Browser theft, Mimikatz*	Short-lived, context-bound tokens; revocation.
Kerberoasting	Request TGS tickets for *SPN* accounts, crack offline (RC4). **Needs valid domain creds.**	Rubeus*, Impacket `GetUserSPNs`, hashcat	Long service-account passwords, gMSA, AES.
AS-REP roasting	Target accounts with *pre-auth disabled*; crack the AS-REP. **Needs no creds.**	Impacket `GetNPUsers`, hashcat	Require Kerberos pre-authentication.
Golden / Silver ticket	Forge a TGT (with the `krbtgt` hash) or a service ticket (with the service hash).	Mimikatz*	Rotate krbtgt twice, protect DCs.
Capture step	Poison LLMNR/NBT-NS to grab *NetNTLMv2* hashes (then crack or relay).	Responder	Disable LLMNR/NBT-NS, enforce SMB signing.
Attack-path mapping	Graph AD relationships to find *who* to target and the route to Domain Admin.	BloodHound	Tier model, prune excessive rights.

** Tool is formally listed under Objective 4.4; shown here because the technique appears in 4.3. The 4.3 official tool list is CME, Responder, hashcat, John, Hydra, BloodHound, Medusa, Burp Suite.*

REMEMBER: possession = identity — **HASH (PTH) · TICKET (PTT) · TOKEN (PASS-THE-TOKEN).** And the classic exam split: **KERBEROAST NEEDS CREDS + TARGETS SPNS; AS-REP NEEDS NEITHER + TARGETS PRE-AUTH-DISABLED ACCOUNTS.**

4 · ABUSE THE PROTOCOL — directory, SSO & MFA

Instead of the password, attack the mechanism that *checks* it.

Attack	How it works	Tool(s)	Fix
LDAP injection	Inject into LDAP filters — "SQLi for directories" — to bypass auth or dump the directory. e.g. `*)(uid=*))(|(uid=*` .	Burp Suite	Input validation, safe/parameterized LDAP filters.
OIDC attacks	Abuse OpenID Connect flows: `redirect_uri` tampering, token/nonce replay, weak ID-token validation.	Burp Suite	Validate redirect URIs, nonce, signatures.
SAML attacks	Forge/alter SAML assertions ("Golden SAML"), XML signature wrapping to impersonate.	Burp, SAML Raider	Validate signatures, protect IdP signing keys.
MFA fatigue	Spam push-approval prompts until the user reflexively taps "approve" (push bombing).	Manual / scripted push	Number matching, limit prompts, phishing-resistant MFA.

REMEMBER: OIDC/SAML = SSO FEDERATION TRUST · LDAP INJECTION = SQLI FOR THE DIRECTORY · MFA FATIGUE = ANNOY-UNTIL-APPROVE.

4.3 Tool Cheat Sheet

Tool	Use it for
Hydra / Medusa	Online password brute-force across many protocols (SSH, RDP, HTTP, SMB…).
CrackMapExec (CME)	Spray, pass-the-hash, and enumerate across SMB/AD at scale.
Responder	Poison LLMNR/NBT-NS to capture NetNTLMv2 hashes.
hashcat / John	Offline hash cracking (modes: straight, rule, mask, hybrid).
BloodHound	Map Active Directory attack paths to Domain Admin.
Burp Suite	Test LDAP injection and OIDC/SAML federation flows.

4.5 — Web Application Attacks

perform web application attacks using the appropriate tools

5 FAMILIES: INJECTION · INCLUSION · BROKEN ACCESS · FORGERY · SERIALIZATION/LOGIC

Web apps wrongly trust three things: user *input* (→ injection & inclusion), the *client* (→ broken access), and *other services* (→ forgery). Classify any web scenario into one of five families and the answer follows.

Family 1 · INJECTION — input becomes code

The #1 web bug family. Attacker input crosses into an interpreter and is executed. All four are the *same root cause* at different interpreters.

Attack	How it works (example)	Tool(s)	Fix
SQL injection	Input hits the DB engine. In-band (union/error) or blind (boolean/time). Ex: `' OR '1'='1' --`	sqlmap, Burp	Parameterized queries; least-priv DB user.
Command injection	Input reaches the OS shell. Ex: `; whoami` or `| id`	Burp, commix	Avoid shell calls; safe APIs; validate.
XSS	Input runs in the browser (stored / reflected / DOM). Ex: `<script>alert(1)</script>`	Burp, ZAP	Output encoding + Content-Security-Policy.
SSTI	Input evaluated by the template engine. Ex: `{{7*7}}` returns `49` → escalate to RCE.	tplmap, Burp	Sandbox templates; never concatenate user input into them.

REMEMBER: name the interpreter — **DB → SQLI · SHELL → COMMAND INJECTION · BROWSER → XSS · TEMPLATE → SSTI.** Best SQLi fix is always *parameterized queries* (not WAF/validation).

Family 2 · INCLUSION & TRAVERSAL — file & path abuse

Make the app read or run files it shouldn't.

Attack	How it works (example)	Tool(s)	Fix
Directory traversal	Escape the web root to read arbitrary files. Ex: `../../../etc/passwd`	Burp, Wfuzz	Canonicalize paths + allowlist.
LFI	Include a file *already on the server*; can chain to RCE (e.g., log poisoning).	Burp, Wfuzz	Disable dynamic includes; allowlist.
RFI	Include a *remote, attacker-hosted* file → direct RCE.	Burp	Disable remote include (`allow_url_include` off).
Web shell	Uploaded/injected script giving command execution over HTTP (e.g., China Chopper, weevely).	Burp, Gobuster	Validate uploads; no-execute upload dir; store outside webroot.

REMEMBER: traversal **READS** files · LFI/RFI **INCLUDE** (and can execute) them · web shell = **PERSISTENT RCE OVER HTTP.** Local file (LFI) vs remote attacker file (RFI).

Family 3 · BROKEN ACCESS & IDENTITY — trust the client

Anything client-side (IDs, tokens, cookies, hidden fields) is attacker-forgeable. Access breaks when the server assumes it isn't.

Attack	How it works (example)	Tool(s)	Fix
IDOR	Change an identifier to reach others' objects. Ex: `/account?id=123` → `124`.	Burp	Per-object authorization, server-side, every request.
JWT manipulation	Tamper the token: `alg:none`, weak HMAC key, or `kid` injection.	Burp, jwt_tool	Verify signature + algorithm; strong keys; expiry.
Session hijacking	Steal, predict, or fixate a session ID (via XSS, sniffing, or fixation).	Burp	Secure + HttpOnly cookies; regenerate session on login.
API abuse	BOLA/IDOR on endpoints, mass assignment, excessive data exposure, missing rate limits.	Postman, Burp	Per-object authz; schema validation; rate limiting.

REMEMBER: client-side data is forgeable — **ENFORCE AUTHORIZATION SERVER-SIDE, PER OBJECT, ON EVERY REQUEST.** "Hide the ID / client-side check" is never the fix.

Family 4 · FORGERY — the confused deputy

Trick a trusted component into acting for you against a target that trusts *it*.

Attack	How it works (example)	Tool(s)	Fix
SSRF	Make the *server* fetch an attacker URL → internal services or cloud metadata (`169.254.169.254`).	Burp	Egress allowlist; block link-local; enforce IMDSv2.
CSRF	Make the *victim's browser* send an authenticated request using their cookies.	Burp	Anti-CSRF token; SameSite cookies.

REMEMBER: who's the confused deputy — **THE SERVER (SSRF)** or **THE VICTIM'S BROWSER (CSRF)?**

Family 5 · SERIALIZATION, LOGIC & INTEGRITY

The "everything else" family — object handling, weak crypto, code execution, and app-layer guessing.

Attack	How it works	Tool(s)	Fix
Deserialization	Feed crafted serialized objects that trigger code execution ("gadget chains").	ysoserial, Burp	Don't deserialize untrusted data; integrity checks.
Collision attack	Two inputs sharing a hash defeat integrity/signatures (weak MD5/SHA-1).	hashcat (concept)	Strong collision-resistant hashes (SHA-256+).
Arbitrary code execution	The end goal many bugs chain toward — running your code on the server.	Metasploit, Burp	Patch; safe input handling; least privilege.
Brute-force (web)	Guess credentials, tokens, or hidden paths *through* the app.	Burp Intruder, Wfuzz, Gobuster	Lockout, MFA, rate limiting.

REMEMBER: deserialization / ACE = **CODE EXECUTION** · collision = **BROKEN HASH MATH** · web brute-force = **GUESSING VIA THE APP.**

4.5 Tool Cheat Sheet

Tool	Use it for
Burp Suite / ZAP	Intercepting proxies — the core web-testing tools (repeat, intrude, fuzz).
sqlmap	Automated SQL-injection detection and exploitation.
Gobuster / DirBuster	Brute-force directories, files, and subdomains (find hidden endpoints).
Wfuzz	Fuzz parameters and paths to discover flaws.
WPScan	WordPress-specific vulnerability scanning.
Postman	Craft and test API requests.
TruffleHog	Find secrets/API keys leaked in repos or responses.

How to Understand & Memorize These Two

- **Learn the two spines first.** 4.3 = *Guess → Crack → Steal → Abuse*. 4.5 = *Injection · Inclusion · Broken Access · Forgery · Serialization*. Everything hangs off these; recall the spine and the items follow.
- **Classify before you answer.** On any scenario, first name the family — that alone eliminates most distractors. "Server fetching an internal URL" → Forgery → SSRF, before you read the options.
- **Turn the examples into recognition flashcards.** Front: `' OR '1'='1` / `../../etc/passwd` / `{{7*7}}` / `alg:none`. Back: the attack name. The exam shows you these and asks what they are.
- **Anchor the tool per job.** 4.5: Burp for almost everything, *sqlmap* for SQLi, *Gobuster* for discovery, *WPScan* for WordPress, *Postman* for APIs. 4.3: *Hydra/Medusa* online, *hashcat/John* offline, *Responder* to capture, *CME* to spray/PtH, *BloodHound* to map.
- **Do the labs.** For 4.5, PortSwigger's free Web Security Academy maps almost one-to-one to this family list. For 4.3, a TryHackMe/HTB Active Directory or password-attacks room cements the Steal/Crack stages.
- **Drill the two highest-value discriminators:** Kerberoasting vs AS-REP roasting (4.3), and SSRF vs CSRF (4.5).

Authorized use only. Every technique and example here is for penetration testing within a defined scope and signed rules of engagement. The example payloads are standard recognition aids for exam study; using any of this against systems you don't own or aren't authorized to test is illegal. As a tester, document what you did and pair each finding with the remediation shown.

CompTIA PenTest+ PT0-003 (Objectives v3.0) · Deep dive on Objectives 4.3 (Authentication Attacks) & 4.5 (Web Application Attacks) — the two densest objectives in Domain 4 (35%). Organized by mechanism family with official tool mappings. Companion to the four Domain 4 overview guides.